CS163 Data Structure Visualizer - Phase 1 Onboarding & Guidelines

Welcome to Phase 1 of our CS163 Data Structures Visualization project!

Our goal for Phase 1 is to build the underlying "engine" of the visualizer using the Model-View-Controller (MVC) pattern. We are not writing the specific Data Structures yet; we are building the framework that will run them, draw them, and animate them seamlessly on both Windows and Mac.

Please read through this guide carefully. Our grading scheme is strict: if we fail basic requirements like using C++, multiple source files, or Git, **the project result is 0 point**. Let's make sure we are all on the same page to avoid messy code and Git merge conflicts.

---

1. Project Requirements & Folder Architecture

Per our project rules, we must use C++ and use `struct` for every data structure. Each struct must be split into two files:

- A header file (`.h`) for the struct definition and function prototypes.
- A source file (`.cpp`) for the function implementation.
- Our source code must also have at least 2 files with a struct and a `main.cpp`.

To keep our project organized and compliant, we have strictly separated headers and source files into an MVC structure:

- `include/` : **Contains ONLY `.h` files.**
  - `Model/` : Data states (`Theme.h` , `Frame.h` , `Timeline.h`).
  - `View/` : Rendering and UI (`Window.h` , `UIManager.h`).
  - `Controller/` : Logic and input (`AppEngine.h` , `Playback.h`).
- `src/` : **Contains ONLY `.cpp` files.** Follows the exact same layout as `include/`.
  - `main.cpp` : The entry point.
- `resources/` : Fonts, icons, and default data files.

> **Warning:** Do not put any `.cpp` files in the `include` folder, or our CMake build will fail.

---

2. Project Setup & Cross-Platform Compilation

We are using CMake to ensure our C++ code compiles perfectly on both Windows and macOS.

Step 1: Clone the Repository

Open your terminal or Git Bash and clone our repository:

```
git clone https://github.com/ndmhuy/DataStructureVisualizer.git
cd DataStructureVisualizer
```

Step 2: Build and Run the Project Method A: The Automated Scripts (Recommended) You do not need to memorize CMake terminal commands. Use the provided execution scripts based on your OS:

For Windows Users: Double-click run.bat in the root folder.

For macOS Users: Open your Terminal, give the script permission by running chmod +x run.command, and then double-click run.command.

Method B: Manual CMake Terminal Commands (Fallback) If the scripts fail or you are using an IDE like VS Code and want to compile manually, run these exact commands from the project root:

Bash

CS163 Data Structure Visualizer — Phase 1

Welcome! This repository contains the Phase 1 framework for the CS163 Data Structure Visualizer. Phase 1 focuses on building the MVC engine (application structure, rendering, playback, and serialization) — data structures themselves will be added in later phases.

This README summarizes project requirements, repository layout, and how to build/run the project on Windows and macOS.

Requirements

- Language: C++
- Project must use `struct` for each data structure; each `struct` should have a header (`.h`) and a source (`.cpp`).
- The codebase must include at least two source files containing `struct`s and a `main.cpp` entry point.

Repository Layout

Top-level layout (important folders):

- `include/` — headers only (`.h` files). Organized by MVC:
  - `include/Model/` — `Theme.h` , `Frame.h` , `Timeline.h`
  - `include/View/` — `Window.h` , `UIManager.h` , `Renderer.h`
  - `include/Controller/` — `AppEngine.h` , `Playback.h`
- `src/` — implementation files (`.cpp`) mirroring `include/` structure. Contains `main.cpp` .
- `resources/` — fonts, icons, example data files used by the application.

Warning: Do NOT put `.cpp` files inside the `include/` folder; CMake expects headers there and mixes will break the build.

Build & Run

We use CMake so the project builds on both Windows and macOS.

1. Clone the repository

```
git clone https://github.com/ndmhuy/DataStructureVisualizer.git
cd DataStructureVisualizer
```

2. Recommended: use the provided scripts

- Windows: run `run.bat` from the repository root.
- macOS: make `run.command` executable and run it:

```
chmod +x run.command
./run.command
```

3. Manual CMake (fallback / IDE workflows)

```
# from project root
mkdir -p build
cd build
cmake ..
cmake --build .
# the executable will be in build/ (or build/Debug/ on some systems)
```

If the build fails, copy the error output and open an issue or create a branch with a failing test so teammates can reproduce the problem.

Git Workflow & Standards

We require use of Git. The TA may inspect commit history and count commits, so please follow these practices:

- Branching
  - Create feature branches off `dev` (not directly off `main`).
  - Suggested prefixes: `feat/`, `fix/`, `docs/`, `refactor/`.
  - Example: `git checkout -b feat/playback-slider`
- Commit messages
  - Use Conventional Commits: `<type>: <short description>`.
  - Examples: `feat: add Playback struct`, `fix: correct renderer memory leak`.
  - Commit often with small, focused changes.
- Pull Requests
  - Push your branch to GitHub and open a PR targeting `dev`.
  - Include a short description and link to any related issues.
  - At least one teammate should review before merging.

Quick Checklist (before submission)

- Build succeeds locally on your OS.
- Project follows `include/` (headers) and `src/` (sources) separation.
- Commits are frequent and follow Conventional Commits.
- Work was developed on a feature branch and merged via PR after peer review.

Help / Next Steps

If you need help building the project or fixing CMake errors, open an issue or create a branch named `fix/cmake-<os>` and push your reproduce steps.

Thanks — let's get the framework stable so Phase 2 (implementing data structures) is straightforward!